Давай вместо заглушек найдем ошибки , ломающие обратный проход . Вот актуальная версия __call2__:

```python
    def __call2__(self, program_text: str):
        autograd_ctx=self._autograd_ctx #захватываем ссылку на autograd
        # 1. Захватываем область видимости из фрейма вызова forward()
        caller_locals = sys._getframe(1).f_locals
        caller_globals = sys._getframe(1).f_globals
        self._captured_locals = caller_locals

        # Сбрасываем кэш захваченных тензоров
        self._captured_inputs.clear()
        self._captured_outputs.clear()

        # Разделяем входную строку по символу переноса "\n"
        raw_lines = program_text.strip().split('\n')
        compiled_statements = []
        compiled_backward_statements = []  # Для накопления строк обратного прохода
        detected_tensors_set = set()

        for line in raw_lines:
            #line=line+"\n"
            cleaned_line = line.strip()+"\n"
            if not cleaned_line or cleaned_line.startswith('#'):
                continue

            # Заменяем все "nncllloc." на "nn_"
            cleaned_line = cleaned_line.replace('nncllloc.', 'nn_')

            # Вплавляем значения скалярных параметров (например, q = 0.23 -> /=0.23)
            # Ищем потенциальные имена переменных в строке
            tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
            for token in tokens:
                if token in ['nn_', 'sigmoid', 'tanh', 'dropout', 'tid']:
                    continue
                # Если токен найден в локальных переменных и это число (int, float)
                if token in caller_locals and isinstance(caller_locals[token], (int, float)):
                    val = caller_locals[token]
                    # Безопасно заменяем имя переменной на её числовое значение
                    cleaned_line = re.sub(r'\b' + token + r'\b', f"{val}", cleaned_line)

            # --- БЛОК ВАЛИДАЦИИ ИНСТРУКЦИИ ---
            if IS_DEBUG:
                print(f"[compiler_debug]строка полсе пред обработки (до выявления токенов)='{cleaned_line}'")
            # Шаг 1: Проверяем наличие хотя бы одного реального тензора в строке
            line_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
            has_tensor = False
            for t in line_tokens:
                if t in caller_locals and isinstance(caller_locals[t], Tensor):
                    has_tensor = True
                    detected_tensors_set.add(t)

            if not has_tensor:
                raise ValueError(f"Ошибка валидации: Строка '{line}' не содержит ни одного тензора.")

            target_tensor_name = None
```

```python
            line_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', cleaned_line)
            has_tensor = False
            for t in line_tokens:
                if t in caller_locals and isinstance(caller_locals[t], Tensor):
                    has_tensor = True
                    detected_tensors_set.add(t)

            if not has_tensor:
                raise ValueError(f"Ошибка валидации: Строка '{line}' не содержит ни одного тензора.")

            target_tensor_name = None

            # Шаг 2: Проверяем наличие знака равенства "="
            if '=' not in cleaned_line:
                # Если знака равенства нет, должно быть строго ++ или --
                if '++' not in cleaned_line and '--' not in cleaned_line:
                    raise SyntaxError(f"Ошибка валидации: Строка '{line}' не содержит знака '=' или инкремента/декремента (++ / --).")

                # Извлекаем имя тензора, к которому применен инкремент/декремент
                match_inc_dec = re.search(r'\b([a-zA-Z_][a-zA-Z0-9_]*)\b\s*(?:\+\+|--)', cleaned_line)
                if match_inc_dec:
                    target_tensor_name = match_inc_dec.group(1)
            else:
                # Шаг 3: Разделяем строку по знаку равенства
                left_part, right_part = cleaned_line.split('=', 1)
                left_part = left_part.strip()
                right_part = right_part.strip()

                # Проверяем комбинированные операторы присваивания (+=, -=, *=, /=, %=)
                # Если перед '=' стоит один из знаков арифметики
                if left_part and left_part[-1] in ['+', '-', '*', '/', '%']:
                    target_tensor_name = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', left_part)[-1]
                else:
                    # Шаг 4: Чистое присваивание '=', проверяем правую часть на валидность данных
                    target_tensor_name = left_part

                    # Извлекаем все буквенные токены из правой части, кроме вызовов функций nn_
                    right_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                    for rt in right_tokens:
                        if rt.startswith('nn_') or rt in ['sigmoid', 'tanh', 'dropout', 'tid']:
                            continue
                        # Если в правой части осталась строка, которая не является тензором и не была вплавлена как число
                        if rt in caller_locals and isinstance(caller_locals[rt], Tensor):
                            continue
                        raise TypeError(f"Ошибка валидации: В правой части выражения '{line}' обнаружено недопустимое строковое значение '{rt}'.")

            if not target_tensor_name:
                raise ValueError(f"Ошибка валидации: Не удалось определить целевой тензор в строке '{line}'.")

            self._captured_outputs.add(target_tensor_name)
            # --- ГЕНЕРАЦИЯ СТРОКИ ДЛЯ BACKWARD ПО ТВОЕМУ ПЛАНУ ---
            if self.is_training:
                # Дублируем очищенную строку (например, Y=nn_tanh(X))
                bwd_line = cleaned_line
```

```
            self._captured_outputs.add(target_tensor_name)
            # --- ГЕНЕРАЦИЯ СТРОКИ ДЛЯ BACKWARD ПО ТВОЕМУ ПЛАНУ ---
            if self.is_training:
                # Дублируем очищенную строку (например, Y=nn_tanh(X))
                bwd_line = cleaned_line

                if "nn_dropout" in bwd_line:
                    # Исключение для dropout: заменяем функцию на nn_dropout_backward,
                    # заменяя первый аргумент X на его градиент grad_X (входной градиент идет вместо X)
                    # Формат: nn_dropout_backward(grad_Y, p, tid, seed)
                    bwd_line = bwd_line.replace("nn_dropout", "nn_dropout_backward")
                    # Строка 454: Добавляем [1] для получения правой части split
                    compiled_bwd = f"if (tid < {target_tensor_name}Size) {{ grad_{target_tensor_name}[tid] += {bwd_line.split('=', 1)[1]}; }}"
                else:
                    match_func = re.search(r'nn_([a-zA-Z0-9_]+)\(', bwd_line)
                    if match_func:
                        func_name = match_func.group(1)
                        bwd_line = bwd_line.replace(f"nn_{func_name}(", f"nn_{func_name}_backward(")

                        # Строка 464: Экранируем круглые скобки r'...'
                        match_args = re.search(r'nn_[a-zA-Z0-9_]+_backward\((.*?)\)', bwd_line)
                        if match_args:
                            inner_arg = match_args.group(1).split(',')[0].strip()
                            bwd_line = bwd_line.replace(")", f", grad_{target_tensor_name})")

                            # Строка 471: Добавляем [1] к split и [tid] к grad_inner_arg
                            compiled_bwd = f"if (tid < {inner_arg}Size) {{ grad_{inner_arg}[tid] += {bwd_line.split('=', 1)[1]}; }}"
                        else:
                            compiled_bwd = f"if (tid < {target_tensor_name}Size) {{ grad_{target_tensor_name}[tid] += grad_{target_tensor_name}[tid]; }}"

                compiled_backward_statements.append(compiled_bwd)
                # === РЕАЛИЗАЦИЯ УМНОГО ВЕКТОРНОГО ФИЛЬТРА ЗАМЕН ДЛЯ ТЕНЗОРОВ ===
                # Набор операторов и символов из вашего ТЗ (включая пробел)
                operators = [' ', '-', '+', '=', ',', '(', '/', '%', ')','\n','{','}']
                #cleaned_line=cleaned_line+'\n'
                # Обходим уникальные имена тензоров, обнаруженные в строке
                if IS_DEBUG:
                    print(f"[debug_compiler]для строки '{cleaned_line}' найденной {len(detected_tensors_set)} тензоров")
                for t in detected_tensors_set:
                    # 1. Формируем вектор пар: имя + оператор/символ
                    pattern_vector = [f"{t}{op}" for op in operators]

                    # 2. Формируем параллельный вектор замен: имя[tid] + оператор/символ
                    replacement_vector = [f"{t}[tid]{op}" for op in operators]

                    # 3. Выполняем алгоритм контекстной замены:
                    # Пока комбинация имя+оператор встречается в строке, производим замену
                    for pattern, replacement in zip(pattern_vector, replacement_vector):
                        cleaned_line = cleaned_line.replace(pattern, replacement)
                cleaned_line=cleaned_line.strip()
                # Оборачиваем валидную строку в Си-конструкцию с проверкой границ
                compiled_statement = f"if (tid < {t}Size) {{ {cleaned_line}; }}"+"\n"
                if IS_DEBUG:
                    print(f"[debug_compiler_log]результат компиляции строки:'{compiled_statement}'\n")
                compiled_statements.append(compiled_statement)

```

А вот здесь проблема видимо и возникает:
Смотрим прямой:

```
        # Шаг 2: Проверяем наличие знака равенства "="
        if '=' not in cleaned_line:
            # Если знака равенства нет, должно быть строго ++ или --
            if '++' not in cleaned_line and '--' not in cleaned_line:
                raise SyntaxError(f"Ошибка валидации: Строка '{line}' не содержит знака '=' или инкремента/декремента (++ / --).")

            # Извлекаем имя тензора, к которому применен инкремент/декремент
            match_inc_dec = re.search(r'\b([a-zA-Z_][a-zA-Z0-9_]*)\b\s*(?:\+\+|--)', cleaned_line)
            if match_inc_dec:
                target_tensor_name = match_inc_dec.group(1)
        else:
            # Шаг 3: Разделяем строку по знаку равенства
            left_part, right_part = cleaned_line.split('=', 1)
            left_part = left_part.strip()
            right_part = right_part.strip()

            # Проверяем комбинированные операторы присваивания (+=, -=, *=, /=, %=)
            # Если перед '=' стоит один из знаков арифметики
            if left_part and left_part[-1] in ['+', '-', '*', '/', '%']:
                target_tensor_name = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', left_part)[-1]
            else:
                # Шаг 4: Чистое присваивание '=', проверяем правую часть на валидность данных
                target_tensor_name = left_part

                # Извлекаем все буквенные токены из правой части, кроме вызовов функций nn_
                right_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                for rt in right_tokens:
                    if rt.startswith('nn_') or rt in ['sigmoid', 'tanh', 'dropout', 'tid']:
                        continue
                    # Если в правой части осталась строка, которая не является тензором и не была вплавлена как число
                    if rt in caller_locals and isinstance(caller_locals[rt], Tensor):
                        continue
                    raise TypeError(f"Ошибка валидации: В правой части выражения '{line}' обнаружено недопустимое строковое значение '{rt}'.")

```

Обратный явно не соответвует,так как не передает обекты над которыми происходит +-*/